Lab Assignment 8: Data Management Using `pandas`, Part 1

DS 6001: Practice and Application of Data Science

Robert Clay Harris jbm2rt

Instructions

Please answer the following questions as completely as possible using text, code, and the results of code as needed. Format your answers in a Jupyter notebook. To receive full credit, make sure you address every part of the problem, and make sure your document is formatted in a clean and professional way.

In this lab, you will be working with the 2017 Workplace Health in America survey which was conducted by the Centers for Disease Control and Prevention. According to the survey's guidence document:

> The Workplace Health in America (WHA) Survey gathered information from a cross-sectional, nationally representative sample of US worksites. The sample was drawn from the Dun & Bradstreet (D&B) database of all private and public employers in the United States with at least 10 employees. Like previous national surveys, the worksite served as the sampling unit rather than the companies or firms to which the worksites belonged. Worksites were selected using a stratified simple random sample (SRS) design, where the primary strata were ten multi-state regions defined by the Centers for Disease Control and Prevention (CDC), plus an additional stratum containing all hospital worksites.

The data contain over 300 features that report the industry and type of company where the respondents are employed, what kind of health insurance and other health programs are offered, and other characteristics of the workplaces including whether employees are allowed to work from home and the gender and age makeup of the workforce. The data are full of interesting information, but in order to make use of the data a great deal of data manipulation is required first.

Problem 0

Import the following libraries:

In [63]:
```
import numpy as np
import pandas as pd
import sidetable
import sqlite3
import warnings
warnings.filterwarnings('ignore')
```

Problem 1

The raw data are stored in an ASCII file on the 2017 Workplace Health in America survey homepage. Load the raw data directly into Python without downloading the data onto your harddrive and display a dataframe with only the 14th, 28th, and 102nd rows of the data. [1 point]

In [64]:
```
url = "https://www.cdc.gov/workplace-health-promotion/media/files/2024/06/whpps_120717.csv"
wha = pd.read_csv(url, sep='~')

wha.iloc[[13, 27, 101]]
```

Out[64]:

	OC1	OC3	HI1	HI2	HI3	HI4	HRA1	HRA1A	HRA1B	HRA1E	...	WL3_05	E1_09	Suppquex	Id	Region	CDC_Region	Indu
13	3	1.0	2.0	3.0	2.0	1.0	1.0	3.0	3.0	1.0	...	NaN	NaN	1.0	1437.0	4.0	6.0	
27	1	3.0	1.0	3.0	1.0	1.0	1.0	2.0	4.0	2.0	...	NaN	NaN	1.0	2501.0	2.0	4.0	
101	2	1.0	1.0	3.0	2.0	1.0	1.0	2.0	4.0	2.0	...	NaN	NaN	2.0	12636.0	4.0	6.0	

3 rows × 301 columns

Problem 2

The data contain 301 columns. Create a new variable in Python's memory to store a working version of the data. In the working version, delete all of the columns except for the following:

- `Industry` : 7 Industry Categories with NAICS codes
- `Size` : 8 Employee Size Categories
- `OC3` Is your organization for profit, non-profit, government?
- `HI1` In general, do you offer full, partial or no payment of premiums for personal health insurance for full-time employees?
- `HI2` Over the past 12 months, were full-time employees asked to pay a larger proportion, smaller proportion or the same proportion of personal health insurance premiums?
- `HI3` : Does your organization offer personal health insurance for your part-time employees?
- `CP1` : Are there health education programs, which focus on skill development and lifestyle behavior change along with information dissemination and awareness building?
- `WL6` : Allow employees to work from home?
- Every column that begins `WD` , expressing the percentage of employees that have certain characteristics at the firm

[1 point]

In [65]:
```
cols_to_keep = [
    'Industry',
    'Size',
    'OC3',
    'HI1',
    'HI2',
    'HI3',
    'CP1',
    'WL6'
] + [col for col in wha.columns if col.startswith('WD')]

working_df = wha[cols_to_keep].copy()

working_df.head()
```

Out[65]:

	Industry	Size	OC3	HI1	HI2	HI3	CP1	WL6	WD1_1	WD1_2	WD2	WD3	WD4	WD5	WD6	WD7
0	7.0	7.0	3.0	2.0	1.0	2.0	1.0	1.0	25.0	20.0	85.0	60.0	40.0	15.0	0.0	22.0
1	7.0	6.0	3.0	2.0	3.0	1.0	1.0	1.0	997.0	997.0	90.0	90.0	997.0	997.0	0.0	997.0
2	7.0	8.0	3.0	1.0	3.0	1.0	1.0	1.0	35.0	4.0	997.0	997.0	40.0	15.0	997.0	997.0
3	7.0	4.0	2.0	1.0	2.0	1.0	2.0	2.0	50.0	15.0	50.0	85.0	75.0	0.0	0.0	997.0
4	7.0	4.0	3.0	1.0	3.0	1.0	1.0	1.0	50.0	40.0	60.0	60.0	40.0	30.0	0.0	28.0

Problem 3

The codebook for the WHA data contain short descriptions of the meaning of each of the columns in the data. Use these descriptions to decide on better and more intuitive names for the columns in the working version of the data, and rename the columns accordingly. [1 point]

In [66]:
```
working_df.rename(columns={
    'OC3': 'org_type',
    'HI1': 'ft_health_ins_payment',
    'HI2': 'ft_health_ins_premium_change',
    'HI3': 'pt_health_ins_offered',
    'CP1': 'health_education_programs',
    'WL6': 'work_from_home_policy',
    'WD1_1': 'pct_under_30',
    'WD1_2': 'pct_60_or_older',
    'WD2': 'pct_female',
    'WD3': 'pct_hourly',
    'WD4': 'pct_non_daytime_shift',
    'WD5': 'pct_remote',
    'WD6': 'pct_unionized',
    'WD7': 'avg_turnover_pct'
}, inplace=True)

print(working_df.columns)
```

```
Index(['Industry', 'Size', 'org_type', 'ft_health_ins_payment',
       'ft_health_ins_premium_change', 'pt_health_ins_offered',
       'health_education_programs', 'work_from_home_policy', 'pct_under_30',
       'pct_60_or_older', 'pct_female', 'pct_hourly', 'pct_non_daytime_shift',
       'pct_remote', 'pct_unionized', 'avg_turnover_pct'],
      dtype='object')
```

Problem 4

Using the codebook and this dictionary of NAICS industrial codes, place descriptive labels on the categories of the industry column in the working data. [1 point]

In [67]:
```
industry_mapping = {
    1: "Agriculture, Mining, Utilities, Construction, & Manufacturing",
    2: "Wholesale, Retail, & Transportation/Warehousing",
    3: "Arts, Entertainment, Recreation, & Accommodation",
    4: "Information, Finance, Real Estate, & Professional Services",
    5: "Education & Non-Hospital Health Care",
    6: "Public Administration",
    7: "Hospital Worksites"
}

working_df["Industry"] = working_df["Industry"].replace(industry_mapping)

working_df[['Industry']].head()
```

Out[67]:

	Industry
0	Hospital Worksites
1	Hospital Worksites
2	Hospital Worksites
3	Hospital Worksites
4	Hospital Worksites

Problem 5

Using the codebook, recode the "size" column to have three categories: "Small" for workplaces with fewer than 100 employees, "Medium" for workplaces with at least 100 but fewer than 500 employees, and "Large" for companies with at least 500 employees. [Note: Python dataframes have an attribute `.size` that reports the space the dataframe takes up in memory. Don't confuse this attribute with the column named "Size" in the raw data.] [1 point]

In [68]:
```
working_df['Size_numeric'] = pd.to_numeric(working_df['Size'], errors='coerce')

# Categories 1, 2, 3 (10–99 employees) - "Small"
# Categories 4, 5 (100–499 employees) - "Medium"
# Categories 6, 7, 8 (500+ employees) - "Large"
size_mapping = {
    1: "Small",
    2: "Small",
    3: "Small",
    4: "Medium",
    5: "Medium",
    6: "Large",
    7: "Large",
    8: "Large"
}

working_df['Size'] = working_df['Size_numeric'].apply(lambda x: size_mapping.get(int(x)) if pd.notna(x) else None)

working_df.drop(columns=['Size_numeric'], inplace=True)

working_df[['Size']].head()
```

Out[68]:

	Size
0	Large
1	Large
2	Large
3	Medium
4	Medium

Problem 6

Use the codebook to write accurate and descriptive labels for each category for each categorical column in the working data. Then apply all of these labels to the data at once. Code "Legitimate Skip", "Don't know", "Refused", and "Blank" as missing values. [2 points]

In [69]:

```
# Mapping for OC3: Organization Type
oc3_mapping = {
    1: "For profit, public",
    2: "For profit, private",
    3: "Non-profit",
    4: "State or local government",
    5: "Federal government",
    6: "Other",
    97: np.nan,   # Don't know
    98: np.nan,   # Refusal
    99: np.nan    # Blank
}

# Mapping for HI1: Health Insurance for full-time employees
hi1_mapping = {
    1: "Full insurance coverage offered",
    2: "Partial insurance coverage offered",
    3: "No insurance coverage offered",
    97: np.nan,   # Don't know
    98: np.nan,   # Refusal
    99: np.nan    # Blank
}

# Mapping for HI2: Change in premium proportion for full-time employees
hi2_mapping = {
    1: "Larger",
    2: "Smaller",
    3: "About the same",
    96: np.nan,   # Legitimate Skip
    97: np.nan,   # Don't know
    98: np.nan,   # Refusal
    99: np.nan    # Blank
}

# Mapping for HI3: Health Insurance for part-time employees
hi3_mapping = {
    1: "Yes",
    2: "No",
    97: np.nan,   # Don't know
    98: np.nan,   # Refusal
    99: np.nan    # Blank
}

# Mapping for CP1: Health Education Programs
cp1_mapping = {
    1: "Yes",
    2: "No",
    97: np.nan,   # Don't know
    98: np.nan    # Refusal
}

# Mapping for WL6: Work-from-home Policy
wl6_mapping = {
    1: "Yes",
    2: "No",
    97: np.nan,   # Don't know
    98: np.nan,   # Refusal
    99: np.nan    # Blank
}
```

```
working_df['org_type'] = working_df['org_type'].replace(oc3_mapping)
working_df['ft_health_ins_payment'] = working_df['ft_health_ins_payment'].replace(hi1_mapping)
working_df['ft_health_ins_premium_change'] = working_df['ft_health_ins_premium_change'].replace(hi2_mapping)
working_df['pt_health_ins_offered'] = working_df['pt_health_ins_offered'].replace(hi3_mapping)
working_df['health_education_programs'] = working_df['health_education_programs'].replace(cp1_mapping)
working_df['work_from_home_policy'] = working_df['work_from_home_policy'].replace(wl6_mapping)

display(working_df[['org_type', 'ft_health_ins_payment', 'ft_health_ins_premium_change', 'pt_health_ins_offered', 'health_educ
```

	org_type	ft_health_ins_payment	ft_health_ins_premium_change	pt_health_ins_offered	health_education_programs	work_from_home_policy
0	Non-profit	Partial insurance coverage offered	Larger	No	Yes	Yes
1	Non-profit	Partial insurance coverage offered	About the same	Yes	Yes	Yes
2	Non-profit	Full insurance coverage offered	About the same	Yes	Yes	Yes
3	For profit, private	Full insurance coverage offered	Smaller	Yes	No	No
4	Non-profit	Full insurance coverage offered	About the same	Yes	Yes	Yes

Problem 7

The features that measure the percent of the workforce with a particular characteristic use the codes 997, 998, and 999 to represent "Don't know", "Refusal", and "Blank/Invalid" respectively. Replace these values with missing values for all of the percentage features at the same time. [1 point]

In [70]:
```
pct_cols = [
    'pct_under_30',
    'pct_60_or_older',
    'pct_female',
    'pct_hourly',
    'pct_non_daytime_shift',
    'pct_remote',
    'pct_unionized',
    'avg_turnover_pct'
]

working_df[pct_cols] = working_df[pct_cols].replace({997: np.nan, 998: np.nan, 999: np.nan})

working_df[pct_cols].head()
```

Out[70]:

	pct_under_30	pct_60_or_older	pct_female	pct_hourly	pct_non_daytime_shift	pct_remote	pct_unionized	avg_turnover_pct
0	25.0	20.0	85.0	60.0	40.0	15.0	0.0	22.0
1	NaN	NaN	90.0	90.0	NaN	NaN	0.0	NaN
2	35.0	4.0	NaN	NaN	40.0	15.0	NaN	NaN
3	50.0	15.0	50.0	85.0	75.0	0.0	0.0	NaN
4	50.0	40.0	60.0	60.0	40.0	30.0	0.0	28.0

Problem 8

Sort the working data by industry in ascending alphabetical order. Within industry categories, sort the rows by size in ascending alphabetical order. Within groups with the same industry and size, sort by percent of the workforce that is under 30 in descending numeric order. [1 point]

In [71]:
```
sorted_df = working_df.sort_values(
    by=['Industry', 'Size', 'pct_under_30'],
    ascending=[True, True, False]
)
```

Problem 9

There is one row in the working data that has a `NaN` value for industry. Delete this row. Use a logical expression, and not the row number. [1 point]

In [72]:
```
working_df = working_df[working_df['Industry'].notna()]

print("Remaining NaN values in Industry:", working_df['Industry'].isna().sum())
```

```
Remaining NaN values in Industry: 0
```

Problem 10

Create a new feature named `gender_balance` that has three categories: "Mostly men" for workplaces with between 0% and 35% female employees, "Balanced" for workplaces with more than 35% and at most 65% female employees, and "Mostly women" for workplaces with more than 65% female employees. [1 point]

In [90]:
```
conditions = [
    (working_df['pct_female'] <= 35),
    ((working_df['pct_female'] > 35) & (working_df['pct_female'] <= 65)),
    (working_df['pct_female'] > 65)
]

choices = ["Mostly men", "Balanced", "Mostly women"]

working_df['gender_balance'] = np.select(conditions, choices, default=pd.NA)

working_df[['pct_female', 'gender_balance']].head()
```

Out[90]:

	pct_female	gender_balance
0	85.0	Mostly women
1	90.0	Mostly women
2	NaN	<NA>
3	50.0	Balanced
4	60.0	Balanced

Problem 11

Change the data type of all categorical features in the working data from "object" to "category". [1 point]

In [80]:
```
categorical_cols = working_df.select_dtypes(include=['object']).columns

working_df[categorical_cols] = working_df[categorical_cols].apply(lambda col: col.astype('category'))

print(working_df.dtypes)
```

```
Industry                    category
Size                        category
org_type                    category
ft_health_ins_payment       category
ft_health_ins_premium_change category
pt_health_ins_offered       category
health_education_programs   category
work_from_home_policy       category
pct_under_30                 float64
pct_60_or_older              float64
pct_female                   float64
pct_hourly                   float64
pct_non_daytime_shift        float64
pct_remote                   float64
pct_unionized                float64
avg_turnover_pct             float64
gender_balance              category
dtype: object
```

Problem 12

Filter the data to only those rows that represent small workplaces that allow employees to work from home. Then report how many of these workplaces offer full insurance, partial insurance, and no insurance. Use a function that reports the percent, cumulative count, and cumulative percent in addition to the counts. [1 point]

In [81]:
```
subset_df = working_df[
    (working_df['Size'] == "Small") &
    (working_df['work_from_home_policy'] == "Yes")
]

insurance_freq = subset_df.stb.freq(['ft_health_ins_payment'])

print(insurance_freq)
```

```
              ft_health_ins_payment  count    percent  cumulative_count  \
0     Full insurance coverage offered    324  46.285714               324
1  Partial insurance coverage offered    310  44.285714               634
2       No insurance coverage offered     66   9.428571               700

   cumulative_percent
0           46.285714
1           90.571429
2          100.000000
```

Problem 13

Anything that can be done in SQL can be done with `pandas`. The next several questions ask you to write `pandas` code to match a given SQL query. But to check that the SQL query and `pandas` code yield the same result, create a new database wsing the `sqlite3` package and input the cleaned WHA data as a table in this database. (See module 6 for a discussion of SQlite in Python.) [1 point]

In [82]:
```
conn = sqlite3.connect("wha_clean.db")

working_df.to_sql("wha", conn, if_exists="replace", index=False)

df_sql = pd.read_sql_query("SELECT * FROM wha LIMIT 5;", conn)
print("Sample data from the 'wha' table:")
print(df_sql)
```

```
Sample data from the 'wha' table:
             Industry    Size             org_type  \
0  Hospital Worksites   Large           Non-profit
1  Hospital Worksites   Large           Non-profit
2  Hospital Worksites   Large           Non-profit
3  Hospital Worksites  Medium  For profit, private
4  Hospital Worksites  Medium           Non-profit

               ft_health_ins_payment ft_health_ins_premium_change  \
0  Partial insurance coverage offered                      Larger
1  Partial insurance coverage offered              About the same
2     Full insurance coverage offered              About the same
3     Full insurance coverage offered                     Smaller
4     Full insurance coverage offered              About the same

  pt_health_ins_offered health_education_programs work_from_home_policy  \
0                    No                       Yes                   Yes
1                   Yes                       Yes                   Yes
2                   Yes                       Yes                   Yes
3                   Yes                        No                    No
4                   Yes                       Yes                   Yes

   pct_under_30  pct_60_or_older  pct_female  pct_hourly  \
0          25.0             20.0        85.0        60.0
1           NaN              NaN        90.0        90.0
2          35.0              4.0         NaN         NaN
3          50.0             15.0        50.0        85.0
4          50.0             40.0        60.0        60.0

   pct_non_daytime_shift  pct_remote  pct_unionized  avg_turnover_pct  \
0                   40.0        15.0            0.0              22.0
1                    NaN         NaN            0.0               NaN
2                   40.0        15.0            NaN               NaN
3                   75.0         0.0            0.0               NaN
4                   40.0        30.0            0.0              28.0

  gender_balance
0   Mostly women
1   Mostly women
2           None
3       Balanced
4       Balanced
SQLite database created and cleaned WHA data inserted successfully.
```

Problem 14

Write `pandas` code that replicates the output of the following SQL code:

```
SELECT size, type, premiums AS insurance, percent_female FROM whpps
WHERE industry = 'Hospitals' AND premium_change='Smaller'
ORDER BY percent_female DESC;
```

For each of these queries, your feature names might be different from the ones listed in the query, depending on the names you chose in problem 3. [2 points]

In [83]:
```
sql_query = """
SELECT
    Size,
    org_type,
    ft_health_ins_payment AS insurance,
    pct_female
FROM wha
WHERE Industry = 'Hospital Worksites'
  AND ft_health_ins_premium_change = 'Smaller'
ORDER BY pct_female DESC;
"""

df_sql = pd.read_sql_query(sql_query, conn)

df_sql
```

Out[83]:

	Size	org_type	insurance	pct_female
0	Medium	Non-profit	Full insurance coverage offered	89.0
1	Large	Non-profit	Partial insurance coverage offered	80.0
2	Large	Non-profit	Partial insurance coverage offered	80.0
3	Small	Non-profit	Full insurance coverage offered	75.0
4	Medium	Non-profit	Partial insurance coverage offered	65.0
5	Medium	For profit, private	Full insurance coverage offered	50.0
6	Medium	None	Partial insurance coverage offered	NaN
7	Medium	Non-profit	Partial insurance coverage offered	NaN
8	Medium	Non-profit	Full insurance coverage offered	NaN
9	Medium	Non-profit	Full insurance coverage offered	NaN
10	Large	Non-profit	Partial insurance coverage offered	NaN

In [84]:
```
filtered_df = working_df[
    (working_df["Industry"] == "Hospital Worksites") &
    (working_df["ft_health_ins_premium_change"] == "Smaller")
]

filtered_df = filtered_df.sort_values(by="pct_female", ascending=False)

result_df = filtered_df[["Size", "org_type", "ft_health_ins_payment", "pct_female"]].copy()
result_df.rename(columns={"ft_health_ins_payment": "insurance"}, inplace=True)

result_df
```

Out[84]:

	Size	org_type	insurance	pct_female
320	Medium	Non-profit	Full insurance coverage offered	89.0
187	Large	Non-profit	Partial insurance coverage offered	80.0
214	Large	Non-profit	Partial insurance coverage offered	80.0
229	Small	Non-profit	Full insurance coverage offered	75.0
191	Medium	Non-profit	Partial insurance coverage offered	65.0
3	Medium	For profit, private	Full insurance coverage offered	50.0
11	Medium	NaN	Partial insurance coverage offered	NaN
48	Medium	Non-profit	Partial insurance coverage offered	NaN
51	Medium	Non-profit	Full insurance coverage offered	NaN
75	Medium	Non-profit	Full insurance coverage offered	NaN
97	Large	Non-profit	Partial insurance coverage offered	NaN

Problem 15

Write `pandas` code that replicates the output of the following SQL code:

```
SELECT industry,
    AVG(percent_female) as percent_female,
    AVG(percent_under30) as percent_under30,
    AVG(percent_over60) as percent_over60
FROM whpps
GROUP BY industry
ORDER BY percent_female DESC;
```

[2 points]

In [85]:

```
sql_query = """
SELECT
    Industry,
    AVG(pct_female) AS pct_female,
    AVG(pct_under_30) AS pct_under_30,
    AVG(pct_60_or_older) AS pct_60_or_older
FROM wha
GROUP BY Industry
ORDER BY AVG(pct_female) DESC;
"""

result_sql = pd.read_sql_query(sql_query, con=conn)
result_sql
```

Out[85]:

	Industry	pct_female	pct_under_30	pct_60_or_older
0	Education & Non-Hospital Health Care	80.657143	25.745665	11.349570
1	Hospital Worksites	76.427027	27.213793	16.489655
2	Arts, Entertainment, Recreation, & Accommodation	53.804416	38.566343	11.544872
3	Information, Finance, Real Estate, & Professio...	50.632184	23.821752	12.465465
4	Public Administration	39.056738	21.015625	15.015385
5	Wholesale, Retail, & Transportation/Warehousing	32.657258	29.108696	12.584034
6	Agriculture, Mining, Utilities, Construction, ...	20.328605	22.257143	10.690355

In [86]:

```
result_pandas = (
    working_df.groupby("Industry", as_index=False)[["pct_female", "pct_under_30", "pct_60_or_older"]]
              .mean()
              .sort_values(by="pct_female", ascending=False)
)

result_pandas
```

Out[86]:

	Industry	pct_female	pct_under_30	pct_60_or_older
2	Education & Non-Hospital Health Care	80.657143	25.745665	11.349570
3	Hospital Worksites	76.427027	27.213793	16.489655
1	Arts, Entertainment, Recreation, & Accommodation	53.804416	38.566343	11.544872
4	Information, Finance, Real Estate, & Professio...	50.632184	23.821752	12.465465
5	Public Administration	39.056738	21.015625	15.015385
6	Wholesale, Retail, & Transportation/Warehousing	32.657258	29.108696	12.584034
0	Agriculture, Mining, Utilities, Construction, ...	20.328605	22.257143	10.690355

Problem 16

Write `pandas` code that replicates the output of the following SQL code:

```
SELECT gender_balance, premiums, COUNT(*)
FROM whpps
GROUP BY gender_balance, premiums
HAVING gender_balance is NOT NULL and premiums is NOT NULL;
```

[2 points]

In [87]:
```
sql_query = """
SELECT gender_balance, ft_health_ins_payment, COUNT(*)
FROM wha
GROUP BY gender_balance, ft_health_ins_payment
HAVING gender_balance is NOT NULL and ft_health_ins_payment is NOT NULL;
"""

result_sql = pd.read_sql_query(sql_query, con=conn)
result_sql
```

Out[87]:

	gender_balance	ft_health_ins_payment	COUNT(*)
0	Balanced	Full insurance coverage offered	226
1	Balanced	No insurance coverage offered	77
2	Balanced	Partial insurance coverage offered	271
3	Mostly men	Full insurance coverage offered	301
4	Mostly men	No insurance coverage offered	91
5	Mostly men	Partial insurance coverage offered	332
6	Mostly women	Full insurance coverage offered	267
7	Mostly women	No insurance coverage offered	107
8	Mostly women	Partial insurance coverage offered	333

In [88]:
```
filtered_df = working_df.dropna(subset=['gender_balance', 'ft_health_ins_payment'])

result_df = filtered_df.groupby(['gender_balance', 'ft_health_ins_payment']).size().reset_index(name='count')

result_df
```

Out[88]:

	gender_balance	ft_health_ins_payment	count
0	Balanced	Full insurance coverage offered	226
1	Balanced	No insurance coverage offered	77
2	Balanced	Partial insurance coverage offered	271
3	Mostly men	Full insurance coverage offered	301
4	Mostly men	No insurance coverage offered	91
5	Mostly men	Partial insurance coverage offered	332
6	Mostly women	Full insurance coverage offered	267
7	Mostly women	No insurance coverage offered	107
8	Mostly women	Partial insurance coverage offered	333

In [89]:
```
conn.close()
```